

Architecture des systèmes d'information

Projet Microservices

Module : Architecture des systèmes d'information
ING2 APP LSI2

Encadrant : M Salim BOUZITOUNA

Présenté par : Armando Lopes De Sousa, Lucas Vong, Bryan Bohec, Julien Weng, Oscar Lin, Benjamin Thibout

17/04/2026

Sommaire

1. Contexte et objectifs	3
2. Architecture globale	4
3. Stack technique	5
4. API Gateway	6
5. Services	7
6. Interaction entre les services	10
7. Frontend React	11
8. Authentification et habilitations	12
9. Configuration	13
10. Déploiement Docker	14
11. Conclusion	15

1. Contexte et objectifs

Contexte

Application de location de logement

Architecture en microservices

Libre choix des technologies

Fonctionnalités demandées :

- Interface personnalisable locataires / propriétaires
- Recherche par type, emplacement, budget
- Gestion des inscriptions et offres
- Réservation de logement
- Suivi des demandes et processus

Objectifs

Rapport technique détaillé :

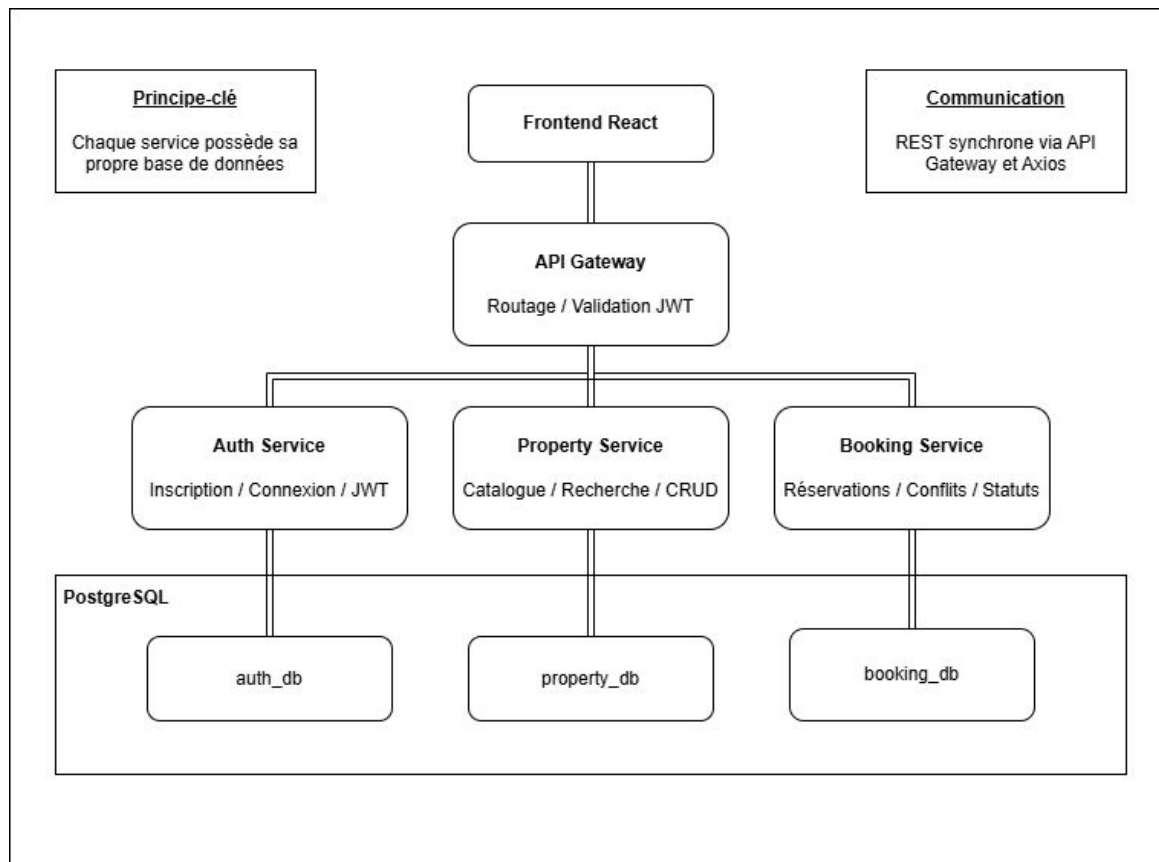
- Conception (découpage, services, interactions)
- Technologies adoptées
- Gestion des interactions
- Configuration et authentification
- Déploiement

Réalisation :

Au moins 2 services implémentés avec toutes les fonctionnalités demandées

Nous avons fait 5 services avec frontend et gateway

2. Architecture globale



3. Stack technique

Frontend	React 19 avec TypeScript, React Router 7, Material-UI 9, Vite, Axios...	
Backend	Node.js 22, Express 5, TypeScript5.9, Prisma 6, Zod 4...	
Base de données	PostgreSQL 17	
Sécurité	JWT, bcrypt, Helmet, CORS, RBAC...	
Déploiement	Docker (8 conteneurs), Healthchecks PostgreSQL...	

4. API Gateway

Point d'entrée unique - Express 5

Routage

/api/auth/* → Auth Service (3001)
/api/properties/* → Property Service (3002)
/api/bookings/* → Booking Service (3003)

Authentification

Verification du token JWT
Extraction des claims (sub, email, role)
Injection des headers :
x-user-id · x-user-role · x-user-email

Sécurité

CORS — origine frontend whitelistee
Timeout 10s sur les requetes proxy

Flux d'une requête authentifiée :

Client → [Authorization: Bearer <JWT>] → API Gateway → jwt.verify() → [x-user-id, x-user-role] → Microservice cible

5. Services - Auth Service

Gestion des utilisateurs & authentification JWT

POST	/auth/register	Inscription
POST	/auth/login	Connexion
GET	/auth/me	Profil de l'utilisateur courant
GET	/auth/users/:id	Recherche utilisateur

Modèle : id, email, password_hash, first_name, last_name, role, created_at, updated_at

Sécurité :

- Mots de passe hachés avec bcrypt - jamais stockés en clair
- JWT signé avec secret partagé, expiration 7 jours, payload : { sub, email, role, iat, exp }
- Token stocké côté client dans localStorage, injecté automatiquement via intercepteur Axios

5. Services - Property Service

Catalogue des logements

GET	/properties	Liste + filtres
GET	/properties/:id	Détails d'une propriété
GET	/properties/:id/availability	Vérification de disponibilité
POST	/properties	Créer une annonce
PUT	/properties/:id	Modifier
DELETE	/properties/:id	Supprimer

Modèle : title, type, address, city, price_per_night, rooms, capacity, owner_id

5. Services - Booking Service

Gestion des réservations

GET	/bookings	Liste des réservations
GET	/bookings/tenant/:id	Réservations d'un locataire
GET	/bookings/owner/:id	Réservations d'un propriétaire
POST	/bookings	Créer une réservation
PATCH	/bookings/:id/status	Modifier le statut
GET	/internal/.../conflicts	Vérification les conflits

Statuts : pending → confirmed → completed
pending → canceled

Logique métier :

- Auto calcul du prix
- Détection des conflits des dates
- RBAC

6. Interaction entre les services

Flux 1

Frontend → API Gateway → Services

Toutes les requêtes HTTP du frontend transitent par le Gateway. Le Gateway valide le JWT, extrait les claims et route vers le bon service. Les endpoints `/internal/` ne sont PAS accessibles depuis le frontend.

Flux 2

Property Service → Booking Service

Vérification de disponibilité : quand un utilisateur consulte la disponibilité d'un logement, le Property Service appelle `GET /internal/bookings/conflicts` sur le Booking Service pour détecter les chevauchements de dates.

Flux 3

Booking Service → Property Service

Création de réservation : le Booking Service valide l'existence de la propriété et récupère le `price_per_night` via `GET /properties/:id` sur le Property Service pour calculer automatiquement le prix total de la réservation.

7. Frontend React

HomePage

Page d'accueil

Présentation de la plateforme

LoginPage

Connexion

Email / Mot de passe

RegisterPage

Inscription

Choix du rôle (tenant/owner)

PropertyPage

Page d'accueil

Filtres : ville, type, prix, capacité

PropertyDetailPage

Détails propriété

Info + formulaire réservation

MyBookingsPage

Mes réservations

Historique et gestion

CreatePropertyPage

Créer annonce

Formulaire de publication

OwnerDashboard

Tableau de bord

Gestion biens + réservations



Public



Locataire



Propriétaire

Communication exclusive avec l'API Gateway via Axios + intercepteur JWT automatique

8. Authentication & Habilitations

Flux JWT

1. L'utilisateur s'inscrit ou se connecte via le Auth Service
2. Le mot de passe est verifie avec `bcrypt.compare()`
3. Un JWT est genere : { sub, email, role } — expiration 7 jours
4. Le frontend stocke le token dans `localStorage`
5. Chaque requete envoie le header `Authorization: Bearer <token>`
6. Le Gateway verifie le JWT et injecte `x-user-id` / `x-user-role` / `x-user-email`

Middlewares de sécurité : `requireAuth`, `requireOwner`, `validationOwner`

RBAC - Matrice des permissions

Action	Locataire	Propriétaire
Voir propriétés	Oui	Oui
Créer propriétés	Non	Oui
Modifier/Suppr. propriétés	Non	Oui
Créer réservation	Oui	Non
Annuler réservation	Oui	Oui
Confirmer réservation	Non	Oui

9. Configuration

Variables d'environnements et validation Zod

Chaque service valide ses variables au démarrage avec Zod.
Si une variable critique est absente ou invalide, le service refuse de démarrer.

Exemple (API Gateway) :

```
PORT: z.coerce.number().default(3000)
JWT_SECRET: z.string().min(12)
AUTH_SERVICE_URL: z.string().url()
PROPERTY_SERVICE_URL: z.string().url()
BOOKING_SERVICE_URL: z.string().url()
```

Résolution DNS

En Docker : résolution via le DNS interne Docker Compose (ex:
`http://auth-service:3001`)

En local : les URLs pointent vers localhost avec les ports correspondants

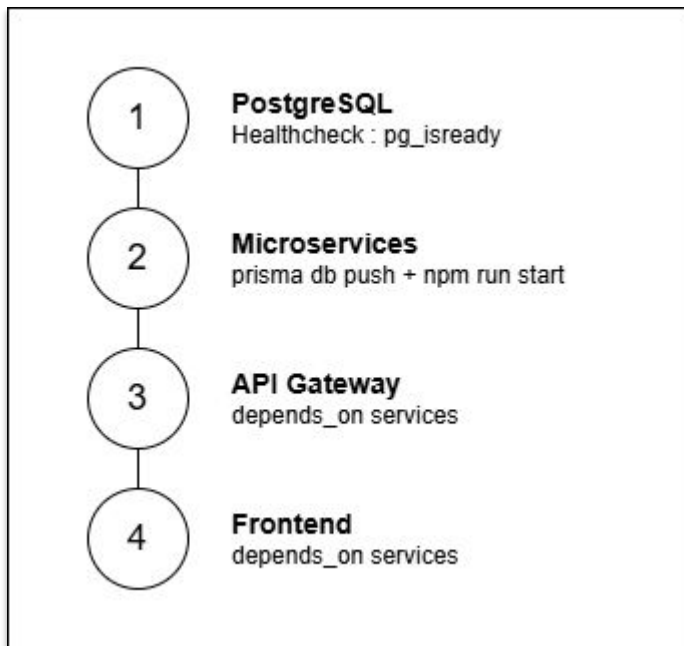
Chaque service connaît ses dépendances via des variables d'environnement (`BOOKING_SERVICE_URL`, etc.)

Fichiers de configuration

Fichiers `.env` pour des variables requises, Docker et la validation Zod. On ajoute à cela un fichier docker compose pour les conteneurs.

10. Déploiement Docker

Ordre de démarrage



Conteneurisation

Image de base : Node.js 22 Alpine (poids minimal)

8 conteneurs : 3 services, 3 BDs, le gateway et le frontend

Lancement complet en une seule commande : `docker compose up --build -d`

11. Conclusion

Architecture microservices : 5 services indépendants, découplés, déployables séparément

Base de données par service : 3 bases isolées, couplage faible

Sécurité multicouche : JWT, bcrypt, etc

Déploiement rapide



Merci pour votre attention

